

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0604	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	G. Hari Prasad	Reg. No.:	192525273
Section / Batch:	B	Date:	29/07/2026

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – DEBUGGING & OPTIMISATION TASK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation
--------------	-----	---	--	---	---

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analyses (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature:	Signature:	Signature:
Date:	Date:	Date:

Q36. Debugging Convex Hull Logic (Collinear Case Mishandling)

The following brute force Convex Hull edge validation logic fails in cases where all remaining points are collinear with a candidate edge. Carefully analyze how collinear points should be handled and explain why the current condition may reject valid hull edges or include incorrect ones. Debug the logic to correctly distinguish between boundary edges and invalid interior segments. Optimize the implementation by reducing repeated orientation checks if possible. Analyze the time complexity of the corrected brute force solution and present the improved code clearly.

```
def is_hull_edge(p1, p2, points):
    for p in points:
        val = orientation(p1, p2, p)
        if val == 0:
            return False # ERROR
        if val < 0:
            return False
        return True
Program python
code:
```

```

def brute_force_convex_hull(points):
    for j in range(i + 1, n):
        if is_hull_edge(points[i], points[j], points):
            hull_edges.append((points[i], points[j]))

    return hull_edges

# Example

points = [
    (0, 0),
    (2, 0),
    (4, 0),
    (4, 3),
    (0, 3)
]

edges = brute_force_convex_hull(points)

print("Hull Edges")

for edge in edges:
    print(edge)

```

Out Put:

```

PS C:\Users\anilk\Desktop\anil> & C:\Users\anilk\AppData\Local\Programs\Python\Python314\python.exe c:/Users/anilk/Desktop/anil/desinganalisis.py
Hull Edges
((0, 0), (2, 0))
((0, 0), (4, 0))
((2, 0), (4, 0))
((4, 0), (4, 3))
((4, 3), (0, 3))
PS C:\Users\anilk\Desktop\anil>

```

Out Put

Q76. Debugging Convex Hull Logic (Endpoint Reversal Not Treated Consistently)

The following brute force Convex Hull helper may produce inconsistent decisions for the same edge when the

two endpoints are supplied in reversed order. Carefully analyze how orientation signs change when the segment

direction is reversed and explain why the current logic can reject a valid hull edge in one direction but accept it in

the other. Debug the code so that edge validation works consistently regardless of endpoint order.

Optimize the

implementation by structuring the side-checking logic more clearly. Analyze the time complexity of the corrected

brute force solution and present the improved logic with proper comments.

```
def is_hull_edge(p1, p2, points):
```

```
    for p in points:
```

```
        val = orientation(p1, p2, p)
```

```
        if val < 0:
```

```
            return False
```

```
    return True
```

```
def brute_force_convex_hull(points):  
    n = len(points)  
    hull_edges = []  
    for i in range(n):  
        for j in range(i + 1, n):  
            if is_hull_edge(points[i], points[j], points):  
                hull_edges.append((points[i], points[j]))  
    return hull_edges
```

```
# Example
```

```
points = [  
    (0, 0),  
    (2, 0),  
    (4, 0),  
    (4, 3),  
    (0, 3)  
]
```

```
edges = brute_force_convex_hull(points)
```

```
print("Hull Edges:")
```

```
for edge in edges:
```

```
    print(edge)
```

Out Put

```
PS C:\Users\anilk\Desktop\anil> & C:\Users\anilk\AppData\Local\Programs\Python\Python314\python.exe c:/Users/an
Full Edges:
((0, 0), (2, 0))
((0, 0), (4, 0))
((0, 0), (0, 3))
((2, 0), (4, 0))
((4, 0), (4, 3))
((4, 3), (0, 3))
PS C:\Users\anilk\Desktop\anil> █
```